

Python Practice Set 1 (Beginners)

Welcome to your first Python practice set!

This set is based on the topics we've covered so far: installation, syntax, variables, typecasting, user input, comments, and operators.

Try to solve each problem on your own before looking at the solution.

Q1: Your First Program

Write a program that prints:

```
Hello, World! Welcome to Python.
```

Q2: Print a Poem

Write a program that prints the following poem using a single `print()` statement:

```
Twinkle, twinkle, little star,  
How I wonder what you are!
```

(Hint: Use `\n` for a new line.)

Q3: Variables & Data Types

Create variables to store: - Your name (string)

- Your age (integer)

- Your height in meters (float)

- A boolean value representing whether you are a student

Print all of them in one line.

Q4: Typecasting Practice

You are given a string:

```
num = "45"
```

Convert it into an integer and add `10` to it. Print the result.

Q5: Taking User Input

Write a program that:

1. Asks the user for their favorite food.

2. Prints:

```
Wow! I also like <food>.
```

Q6: Simple Calculator

Write a program that:

1. Takes two numbers as input from the user.

2. Prints their **sum**, **difference**, **product**, and **quotient**.
-

Q7: Escape Sequences

Print the following output using escape sequences:

```
Harry said, "Python is awesome!"  
This is on a new line.  
This is a tab ->     <- here
```

Q8: Operator Challenge

Write a program that:

1. Takes an integer as input from the user.
 2. Prints the **square** and **cube** of that number.
-

Q9: Quick Quiz (True/False)

Mark **True** or **False**:

1. Python code must always end with a semicolon `;`.
2. The `#` symbol is used for comments in Python.
3. `"123"` and `123` are the same in Python.
4. The `*` operator is used for multiplication.
5. `\n` creates a new line.
6. Variables in Python can start with numbers.
7. `int("10") + 5` gives `15`.

Python Conditionals & Loops - Practice Set

This practice set is based on the topics we've covered so far:

If-Else Conditional Statements, Match Case Statements, For Loops, While Loops, and Break/Continue/Pass Statements.

Use these exercises to practice and solidify your understanding.

1. If-Else Conditional Statements

1. Write a program that asks the user for a number and prints whether it is **positive, negative, or zero**.
 2. Create a program that checks if a person is eligible to vote (age ≥ 18).
 3. Write a program that takes a number from the user and prints "Even" if it is even, otherwise "Odd".
-

2. Match Case Statements

1. Ask the user to enter a day number (1–7) and print the corresponding day of the week using `match case`.
 2. Write a program using `match case` that simulates a simple calculator.
 1. Ask the user for two numbers and an operation (+, -, *, /).
 2. Perform the operation using `match case`.
-

3. For Loops

1. Print numbers from **1 to 10** using a `for` loop.
2. Print the multiplication table of a number (entered by user).
3. Calculate the sum of all numbers from 1 to 100 using a `for` loop.
4. Print the following pattern using a `for` loop:

```
*  
**  
***  
****
```

4. While Loops

1. Print numbers from **1 to 10** using a `while` loop.
2. Write a program that keeps asking the user to enter a password until they enter the correct one.
3. Use a `while` loop to reverse a given number (e.g., 123 → 321).

5. Break, Continue, and Pass Statements

1. Use a `for` loop to print numbers from 1 to 10, but stop the loop if the number is 7 (use `break`).
2. Print numbers from 1 to 10, skipping the number 5 (use `continue`).
3. Write a loop that goes through numbers 1 to 5, but does nothing for number 3 (use `pass`).

Python Strings – Practice Set

This practice set is based on the topics we've covered so far:

- Strings in Python
- String Slicing and Indexing
- String Methods and Functions
- String Formatting and f-Strings
- Strings (2 questions)
- String Manipulation

Use these exercises to strengthen your understanding of strings and become confident in manipulating them.

1. Basic String Operations

1. Create a string variable `name` with your full name. Print:
 1. The first character
 2. The last character
 3. The length of the string
 2. Concatenate two strings: `"Hello"` and `"World"` with a space in between.
-

2. String Slicing and Indexing

1. Given `text = "Python Programming"`, do the following:
 1. Print the first 6 characters
 2. Print the last 6 characters

3. Print every second character from the string
 2. Reverse the string `text` using slicing.
-

3. String Methods and Functions

1. Take the string `" i love python programming "` and:
 1. Remove extra spaces from both ends
 2. Convert it to title case
 3. Count how many times `"o"` appears
 2. Check if the string `"123abc"` is alphanumeric.
-

4. String Formatting and f-Strings

1. Using `format()`, create a sentence:
`"My name is John and I am 25 years old."`
by passing `"John"` and `25` as variables.
 2. Do the same using f-strings.
-

5. String Manipulation Challenges

1. Given `sentence = "Coding in Python is fun"`, replace `"fun"` with `"awesome"` and print it.
 2. Find the index of the word `"Python"` in `sentence`.
 3. Convert the entire `sentence` to uppercase and print it.
-

6. Bonus Questions

1. Write a program that counts how many vowels are in a given string.
2. Take a user input string and check if it is a palindrome (same forwards and backwards).

Python Functions & Modules – Practice Set

This set is based on the topics we've covered so far:

- Defining Functions in Python
- Function Arguments & Return Values
- Lambda Functions in Python
- Recursion in Python
- Modules and Pip – Using External Libraries
- Variable Scope and Docstrings

These exercises will help you get hands-on practice with writing functions, understanding arguments, using recursion, and working with external libraries.

1. Defining Functions

1. Write a function `greet()` that prints `"Hello, Python Learner!"` when called.
 2. Write a function `square(num)` that returns the square of a given number. Test it with different numbers.
-

2. Function Arguments & Return Values

1. Write a function `full_name(first, last)` that takes first name and last name as parameters and returns a single string in the format `"First Last"`.

2. Write a function `calculate_area(length, width=10)` that returns the area of a rectangle. Test it by calling the function with:

1. Both `length` and `width`
 2. Only `length` (use default width)
-

3. Lambda Functions

1. Write a lambda function that adds two numbers and test it.
 2. Create a list `[1, 2, 3, 4, 5]` and use `map()` with a lambda function to get their squares.
-

4. Recursion in Python

1. Write a recursive function `factorial(n)` that returns the factorial of a number.
 2. Write a recursive function `sum_of_digits(n)` that returns the sum of all digits of a given number.
-

5. Modules and Pip – Using External Libraries

1. Import the `math` module and use it to:
 1. Find the square root of 144
 2. Calculate `sin(90°)` (hint: use `math.radians()`)
 2. Install and import the `requests` module (if available) and use it to fetch data from `"https://api.github.com"` .
-

6. Variable Scope and Docstrings

1. Write a function `increment()` that has a local variable `counter` initialized to `0` and increments it by `1` each time it is called. Observe whether the value persists across function calls.
 2. Write a function `multiply(a, b)` that has a proper **docstring** explaining what it does. Then use `help(multiply)` to display the docstring.
-

7. Bonus Challenges

1. Write a recursive function `fibonacci(n)` that prints the first `n` Fibonacci numbers.
2. Write a function `safe_divide(a, b)` that returns the result of `a / b`, but returns `"Cannot divide by zero"` if `b` is `0`.
3. Create a small module `my_utils.py` with a function `is_even(n)` that returns `True` if `n` is even. Import and use it in another Python file.

Python Collections – Practice Set

This set is based on the topics we've covered so far:

- Introduction to Lists
- List Methods
- Tuples and Operations on Tuples
- Sets and Set Methods
- Dictionaries and Dictionary Methods

These exercises will give you hands-on practice with Python's most important data structures.

1. Introduction to Lists

1. Create a list `fruits = ["apple", "banana", "cherry"]` .

1. Print the first fruit.
2. Replace `"banana"` with `"orange"` .
3. Print the length of the list.

2. Create a list of numbers from `1` to `10` .

1. Print the first three numbers using slicing.
 2. Print the last three numbers using slicing.
-

2. List Methods

1. Start with `numbers = [5, 2, 9, 1, 7]` and do the following:

1. Sort the list in ascending order.
2. Append the number `10` to the list.

3. Remove the number `2` from the list.
 2. Create a list `names = ["Alice", "Bob", "Charlie"]` and use the `insert()` method to add `"David"` at index `1`.
-

3. Tuples and Operations on Tuples

1. Create a tuple `coordinates = (10, 20)` and print both elements.
 2. Try to modify the tuple by setting `coordinates[0] = 50` — note what happens.
 3. Convert the tuple to a list, change its first element to `50`, and convert it back to a tuple.
-

4. Sets and Set Methods

1. Create a set `my_set = {1, 2, 3, 3, 4}` and print it. (What happens to duplicate `3`?)
 2. Add `5` to the set, remove `2`, and check if `4` is in the set.
 3. Create two sets:
 1. `a = {1, 2, 3}`
 2. `b = {3, 4, 5}`Find their:
 3. Union
 4. Intersection
 5. Difference (`a - b`)
-

5. Dictionaries and Dictionary Methods

1. Create a dictionary `student = {"name": "John", "age": 20, "grade": "A"}` and:

1. Print the value of `"name"` .
2. Change `"grade"` to `"A+"` .
3. Add a new key `"city"` with value `"Delhi"` .

2. Create a dictionary of three friends and their phone numbers. Use:

1. `keys()` to get all names
2. `values()` to get all numbers
3. `items()` to loop over key-value pairs and print them

6. Bonus Challenges

1. Write a program that takes a list of numbers and removes all duplicates using a set.
2. Given a dictionary of products and their prices, find the product with the highest price.
3. Write a program that merges two dictionaries into one.

Python OOP – Practice Set

This set is based on the topics we've covered so far:

- Introduction to OOP
- Classes and Objects in Python
- Constructors in Python
- Instance and Class Attributes
- Inheritance and Polymorphism
- Method Overriding and Operator Overloading

These are simple starter questions to get you comfortable with object-oriented programming.

1. Create a Simple Class and Object

Create a class `Car` with a method `drive()` that prints `"Car is moving"`.
Create an object of `Car` and call `drive()`.

2. Constructor and Attributes

Create a class `Person` with a constructor (`__init__`) that accepts `name` and `age` as arguments and stores them as instance attributes.
Create an object and print the person's name and age.

3. Simple Inheritance

Create a base class `Animal` with a method `sound()` that prints `"Some sound"` .

Create a derived class `Dog` that overrides `sound()` to print `"Bark!"` .

Create an object of `Dog` and call `sound()` .

Python Advanced Concepts – Practice Set

This set is based on the topics we've covered so far:

- Decorators in Python
- Getters and Setters
- Static & Class Methods
- Magic/Dunder Methods
- Exception Handling and Custom Errors
- `map()`, `filter()`, and `reduce()`
- Walrus Operator
- `args` and `kwargs`

These exercises are designed to take your Python skills to the next level by practicing object-oriented and functional programming features.

1. Decorators in Python

1. Write a decorator `logger` that prints `"Function is being called"` before the function runs. Use it to decorate a function `say_hello()` that prints `"Hello!"`.
 2. Write a decorator `timer` that calculates how long a function takes to execute. Test it with a function that sums numbers from 1 to 1,000,000.
-

2. Getters and Setters

1. Create a class `Employee` with a private attribute `_salary`.
 1. Use `@property` to define a getter for `salary`.
 2. Use `@salary.setter` to prevent setting negative values (print a warning instead).
 3. Create an object and test by setting positive and negative salaries.
-

3. Static & Class Methods

1. Create a class `MathUtils` with:
 1. A `@staticmethod` called `add(a, b)` that returns `a + b`.
 2. A `@classmethod` called `description(cls)` that prints `"This is a utility class for math operations."`
 2. Call both methods without creating an object.
-

4. Magic/Dunder Methods

1. Create a class `Book` with attributes `title` and `author`.
 1. Implement `__str__()` so that printing the object displays `"Title by Author"`.
 2. Implement `__len__()` so that `len(book)` returns the length of the title.
 2. Create two `Book` objects and test these methods.
-

5. Exception Handling and Custom Errors

1. Write a program that asks the user to enter a number and handles:
 1. `ValueError` if the input is not a number
 2. `ZeroDivisionError` if you try to divide by zero
 2. Create a custom exception `NegativeNumberError` and raise it when the user enters a negative number.
-

6. map(), filter(), and reduce()

1. Use `map()` to convert `[1, 2, 3, 4, 5]` into their cubes.
 2. Use `filter()` to get only even numbers from `[10, 11, 12, 13, 14]` .
 3. Use `reduce()` from `functools` to find the product of all elements in `[1, 2, 3, 4]` .
-

7. Walrus Operator

1. Use the walrus operator to read input until the user enters `"quit"` . Print each input as it is entered.
 2. Use the walrus operator in a list comprehension to store lengths of words from `["python", "rocks", "ai"]` in a list while filtering out words shorter than 4 characters.
-

8. *args and **kwargs

1. Write a function `sum_all(*args)` that accepts any number of integers and returns their sum.
2. Write a function `print_details(**kwargs)` that prints key-value pairs passed as arguments, for example:

```
print_details(name="Alice", age=25, city="Delhi")  
# Output:  
# name: Alice  
# age: 25  
# city: Delhi
```

9. Bonus Challenges

1. Combine a decorator with `*args` and `**kwargs` support so it can wrap any function regardless of its parameters.
2. Implement `__add__` in a `Vector` class so that adding two `Vector` objects returns a new `Vector` with summed components.
3. Create a small program where invalid user input raises a custom exception, logs the error, and continues execution instead of crashing.

Python File Handling & Utilities – Practice Set

This set is based on the topics we've covered so far:

- File I/O in Python
- Read, Write, and Append Files
- OS and Shutil Modules in Python
- Creating Command Line Utilities

These exercises will help you practice working with files, directories, and building simple command-line tools.

1. File I/O Basics

1. Create a text file `notes.txt` using Python and write `"Learning Python is fun!"` into it.
 2. Open `notes.txt`, read its content, and print it to the console.
-

2. Read, Write, and Append Files

1. Write a program that writes three lines of text to a file `tasks.txt`.
 2. Open `tasks.txt` in **append mode** and add a new line `"Task Completed!"`.
 3. Read the file and print all lines as a list using `readlines()`.
-

3. OS and Shutil Modules

1. Use the `os` module to:
 1. Print the current working directory
 2. List all files and folders in the current directory
 3. Create a new folder `my_folder`
2. Use the `shutil` module to:
 1. Copy a file from one folder to another
 2. Move a file to a new folder
 3. Delete a file (careful: irreversible!)

4. Creating Command Line Utilities

1. Write a small script `count_lines.py` that takes a filename as input and prints how many lines are in the file.

Example usage:

```
python count_lines.py tasks.txt
# Output: Number of lines: 4
```

2. Write a command-line utility `search_word.py` that takes two arguments:
 1. A filename
 2. A word to search and prints how many times the word appears in the file.

5. Bonus Challenges

1. Write a program that reads a file and creates another file with **all words converted to uppercase**.

2. Create a script that deletes all `.tmp` files from the current directory using `os` and `os.remove()` .
3. Write a Python command-line tool that takes a folder name and prints the total size of all files inside it (use `os.path.getsize()`).